



Université numérique
CHEIKH HAMIDOU KANE

Communication Client-Serveur

Présenté par : DJIGA SENE

Contenu de la séquence

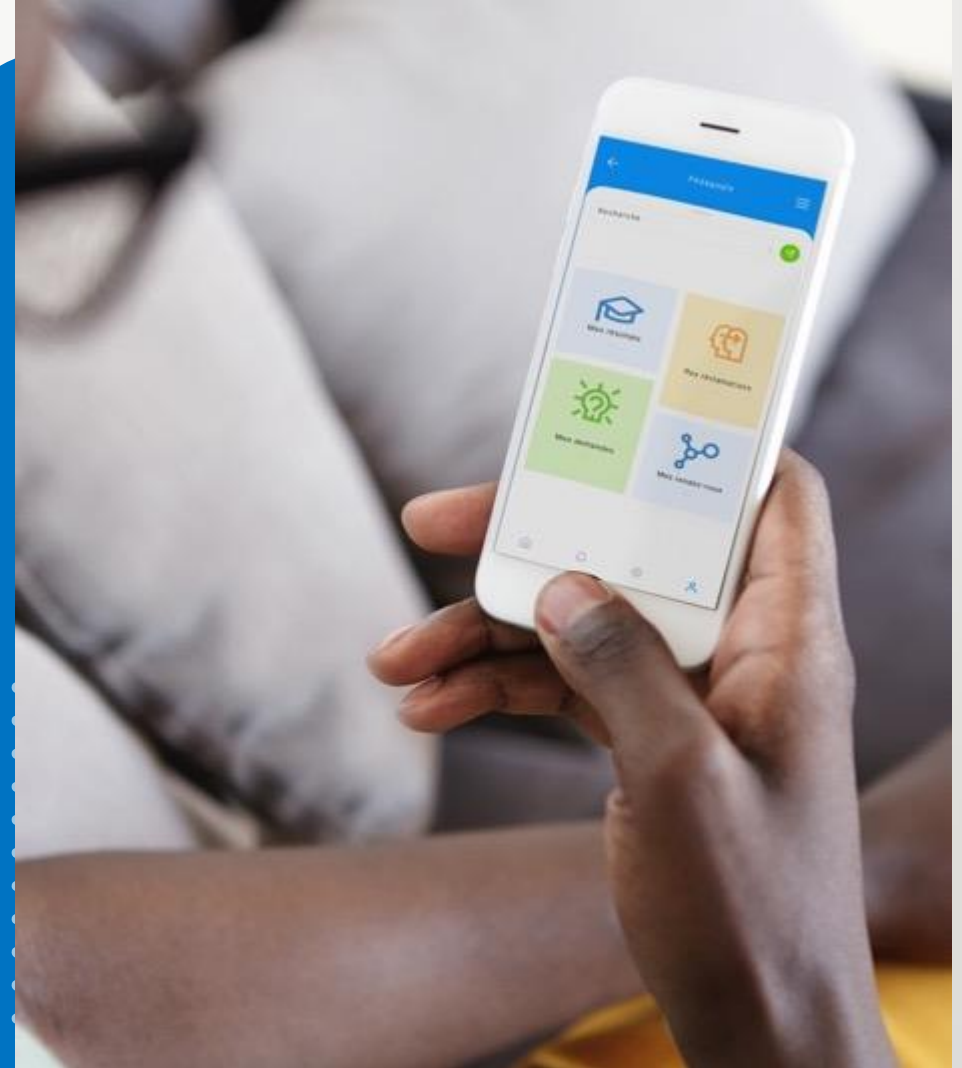
I. Aperçu des frameworks Front-end

II. Approches RESTful et GraphQL

III. Consommation des services Web

IV. Syntaxe des Requêtes GraphQL

V. Avantages de GraphQL



Objectifs spécifiques

A la fin de cette sequence, l'apprenant devrait avoir la capacité de:

- Comprendre les principaux frameworks Front-end utilisés dans le développement web modern et leurs caractéristiques.
- Explorer les deux approches principales pour la creation et la consommation de services Web, en mettant l'accent sur les concepts clés et les differences entre RESTful et GraphQL.
- Apprendre à consommer les services Web RESTful dans les applications Front-end.
- Se familiariser avec la syntaxe et la structure des requêtes GraphQL, y compris les operations, les champs, les alias, les fragments.
- Comprendre les avantages et les inconvénients de l'approche RESTful dans le développement d'API Web.
- Explorer les avantages uniques de GraphQL par rapport à l'approche RESTful.

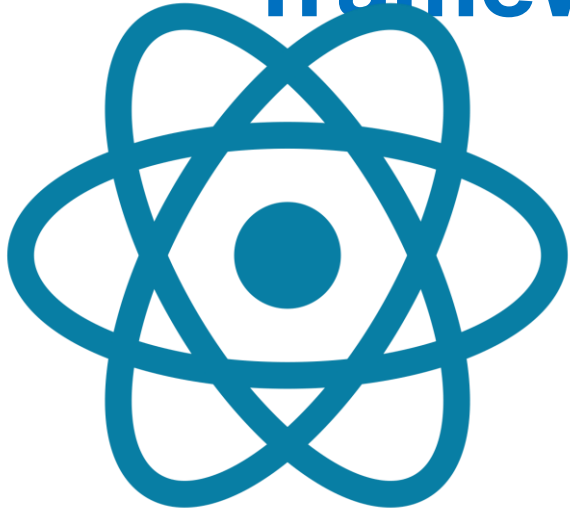


Aperçu des frameworks Front-end

Introduction

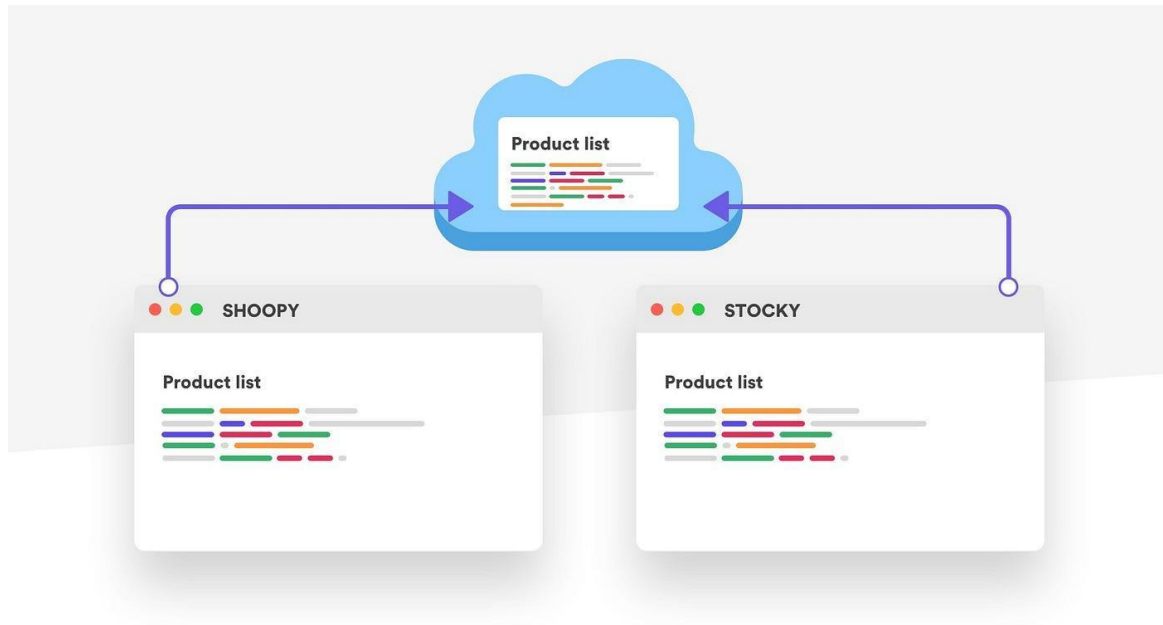


Vue d'ensemble des frameworks Front-end

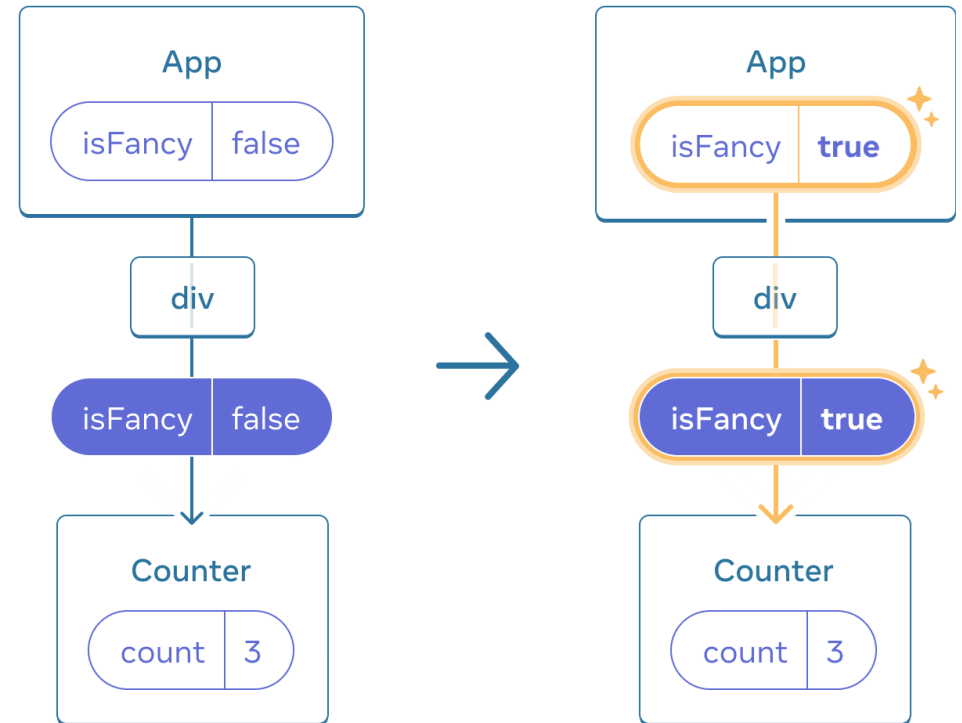


REACT.JS

Réutilisation des



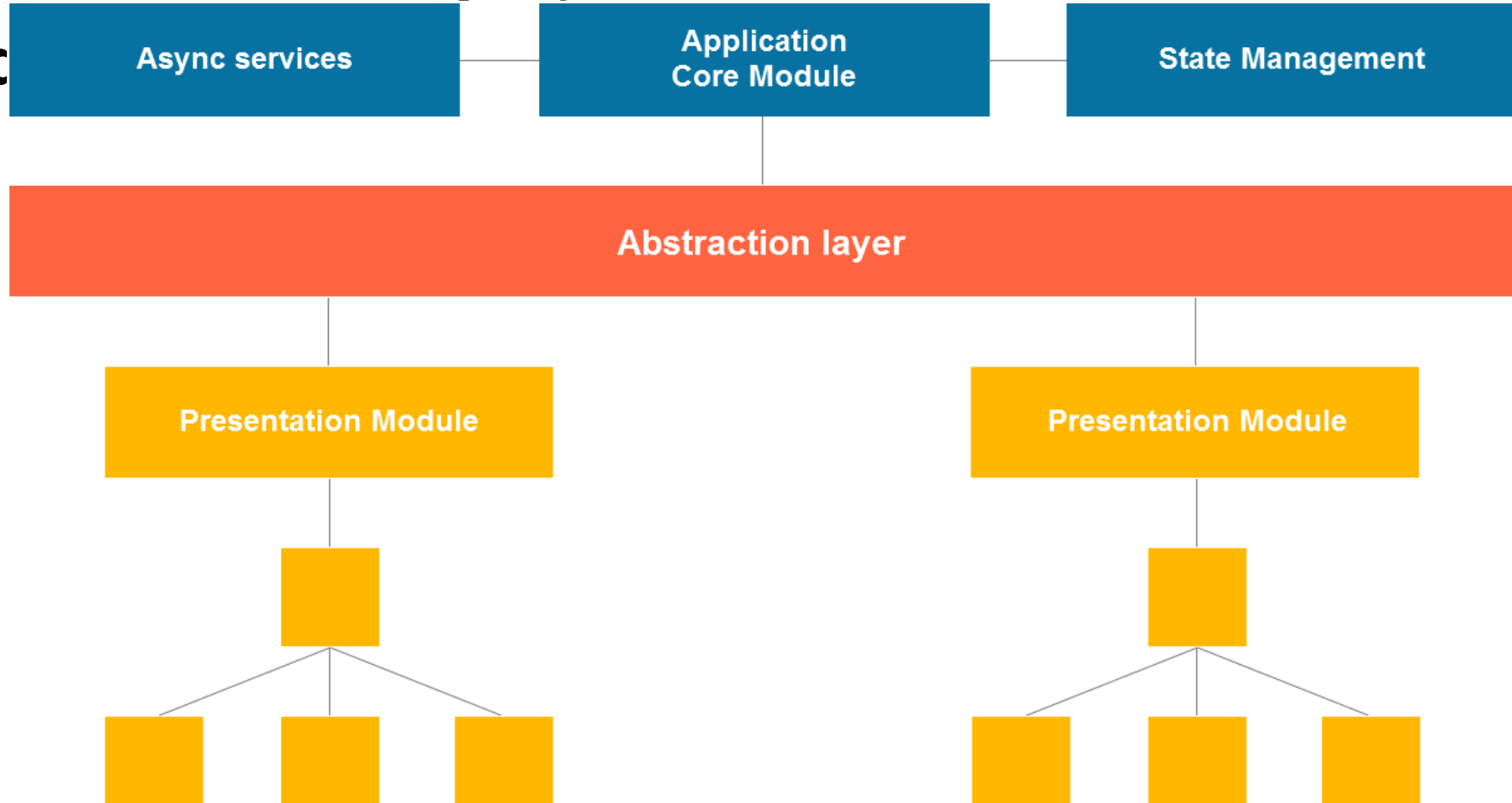
Manipulation d'état



ANGULAR

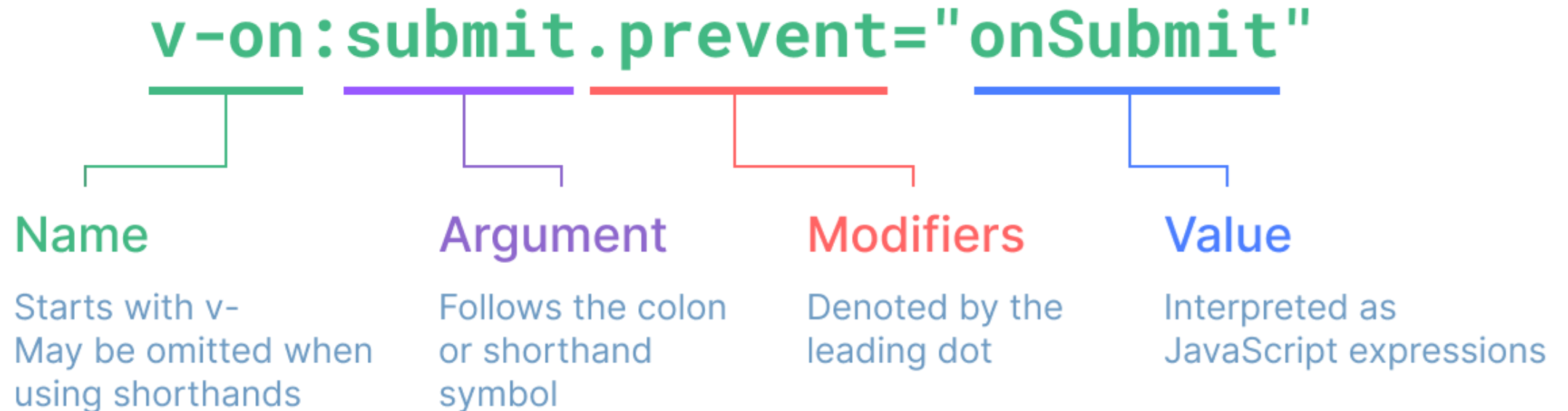
Structuration des projets & utilisation des

mod



VUE.JS

- **Simplicité de la syntaxe**
- **Intégration fluide avec d'autres bibliothèques**



EMBER.JS

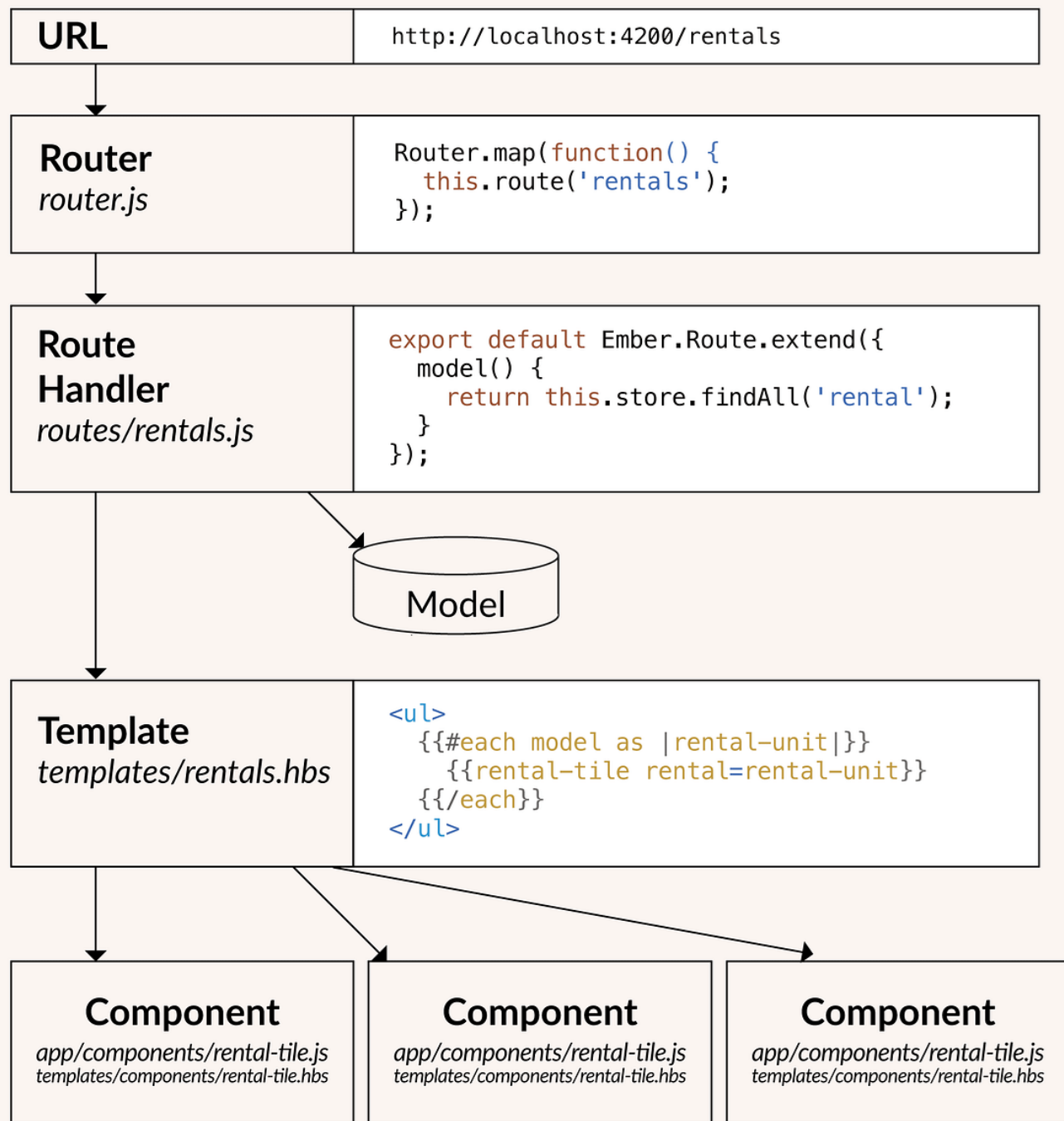
- **Systeme de routage**
- **Gestion des modèles de données**

Maps:
-URL to
a route

Loads:
-a template
-a model

Persists to:
-web server

Loads:
-components
Accesses:
-model data



Comparaison des frameworks

Tableau comparatif des caractéristiques des différents frameworks

Caractéristiques	React	Angular	Vue.js	Ember.js
Langage principal	JavaScript	TypeScript	JavaScript	JavaScript
Taille de la bibliothèque	Légère	Lourde	Légère	Moyenne
Composition des composants	JSX	Templates HTML	Templates HTML	Handlebars
Communauté de support	Très grande	Grande	Grande	Moyenne
Performance	Très rapide	Variable	Rapide	Rapide
Flexibilité	Elevée	Moyenne	Elevée	Moyen
Popularité	Très populaire	Populaire	Populaire	Moins populaire

Choix d'un framework

Le choix d'un framework dépend de plusieurs

facteurs:

- Expérience de l'équipe
- Fonctionnalités requises
- Performances attendues
- Popularité et communauté
- Réactivité de l'interface utilisateur et la gestion de la mémoire
- Flexibilité et extensibilité



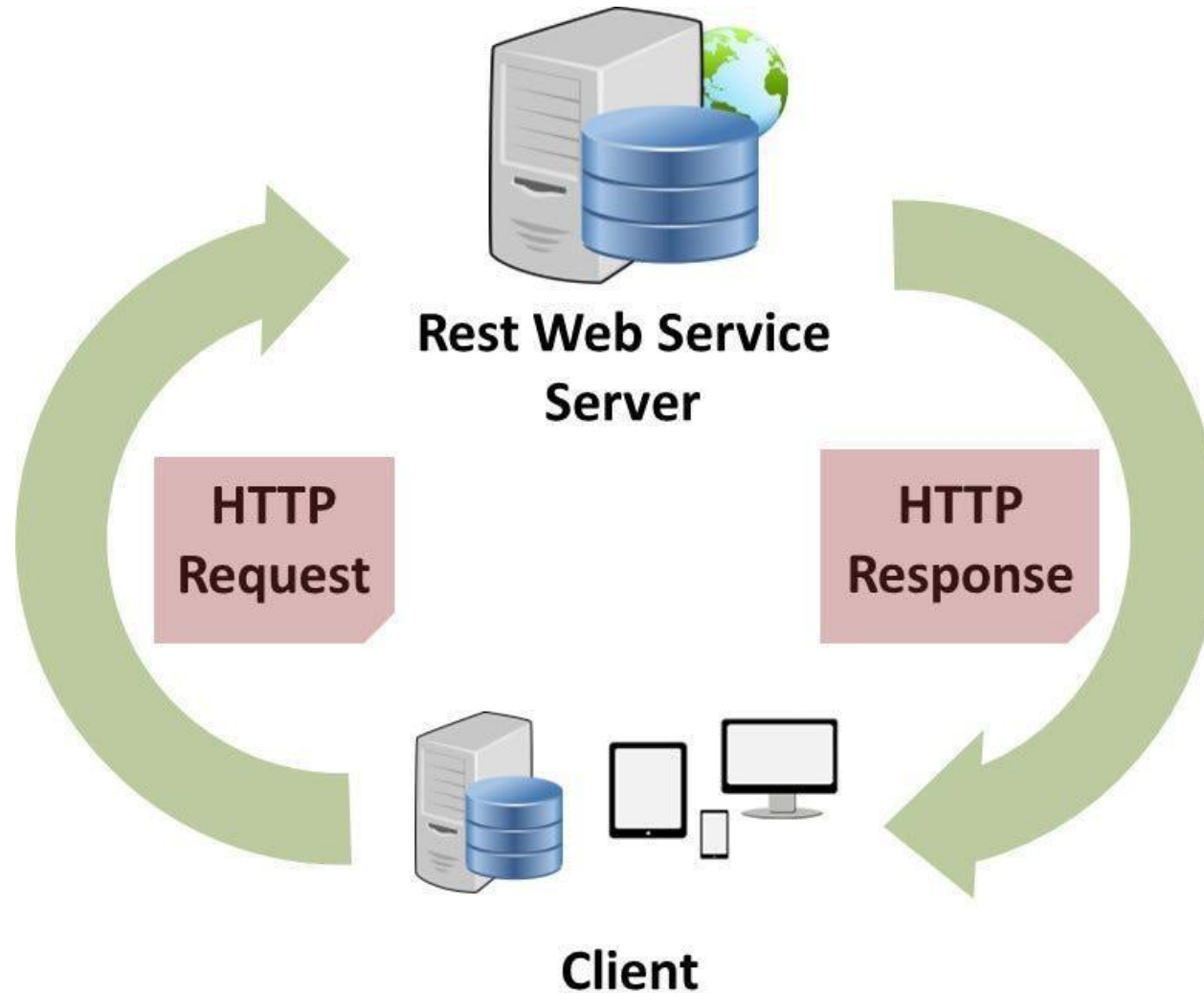
Approches RESTful et GraphQL



Introduction



Approche RESTful



Principes RESTful

- **Architecture client-serveur**
- **Etat de l'application**
- **Interactions sans état**
- **Interface uniforme**
- **Système de couche**
- **Code à la demande**

Approche GraphQL

GraphQL Key Components

```
graph TD; Root[GraphQL Key Components] --- Query[Query]; Root --- Resolver[Resolver]; Root --- Schema[Schema]; Query --- Fields[Fields]; Query --- Arguments[Arguments];
```

Query

Resolver

Schema

Fields

Arguments

Guru99.com

Principes GraphQL – 1/2

Typage fort

Un système de typage fort pour définir la structure des données et garantir la coherence des données échangées entre le client et le serveur.

Interrogation basée sur le graphe

Les clients peuvent spécifier les données exactes dont ils ont besoin en définissant un graphe de requête, ce qui permet de minimizer les surcharges reseau et de récupérer uniquement les données nécessaires.

Mutations

La prise en charge de la mutation pour effectuer des operations de creation, de mise à jour de suppression de données côté serveur, offrant ainsi une interface complète pour interagir avec la source de données.

Abonnements

La possibilité de créer des abonnements permettant aux clients de recevoir des mises à jour en temps reel lorsqu'une condition spécifique ou un événement se produit du côté serveur.

Principes GraphQL – 2/2

Flexibilité des requêtes

Les requêtes GraphQL sont hautement flexibles et peuvent être adaptées aux besoins spécifiques du client en utilisant des arguments, des variables, des fragments et d'autres fonctionnalités avancées.

Découverte automatique du schéma

L'exposition automatique d'un schéma décrivant toutes les fonctionnalités disponibles, ce qui permet aux clients de découvrir facilement les capacités de l'API sans avoir besoin d'une documentation supplémentaire.

Exemple de requête GraphQL

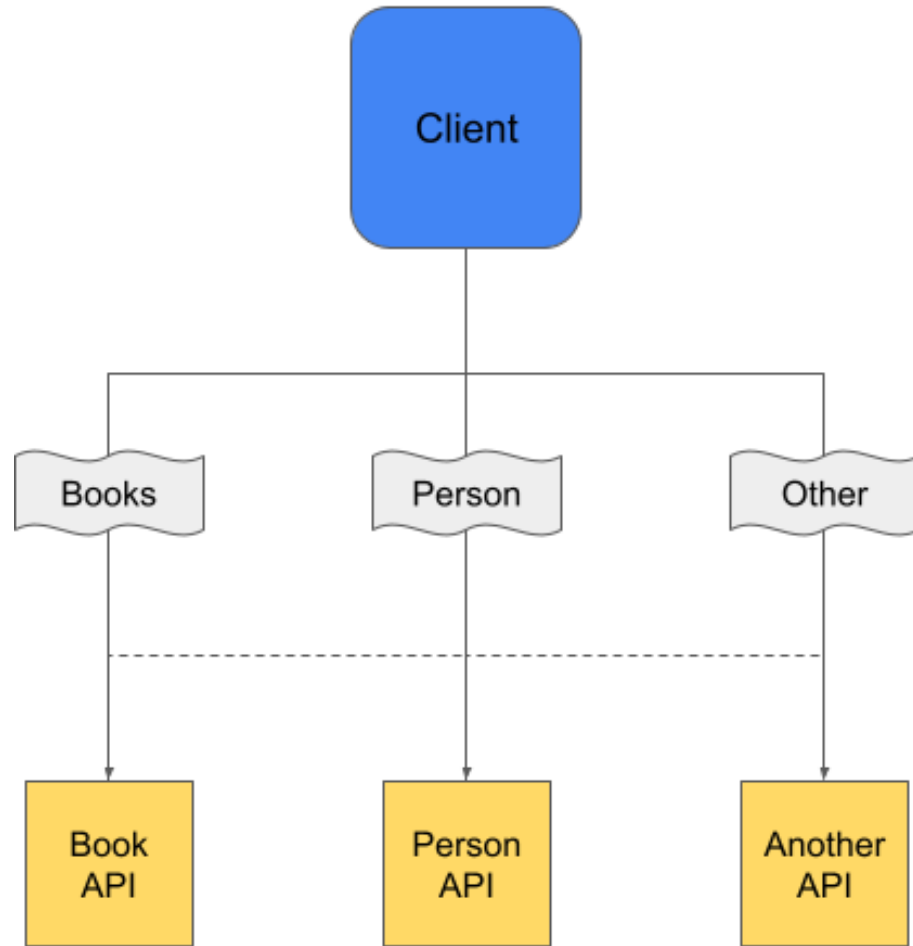
GraphQL: Get 10 Posts w/ Author & Company

```
query {  
  posts(first: "10") {  
    id  
    title  
    author {  
      id  
      firstName  
      company {  
        id  
        companyName  
      }  
    }  
  }  
}
```

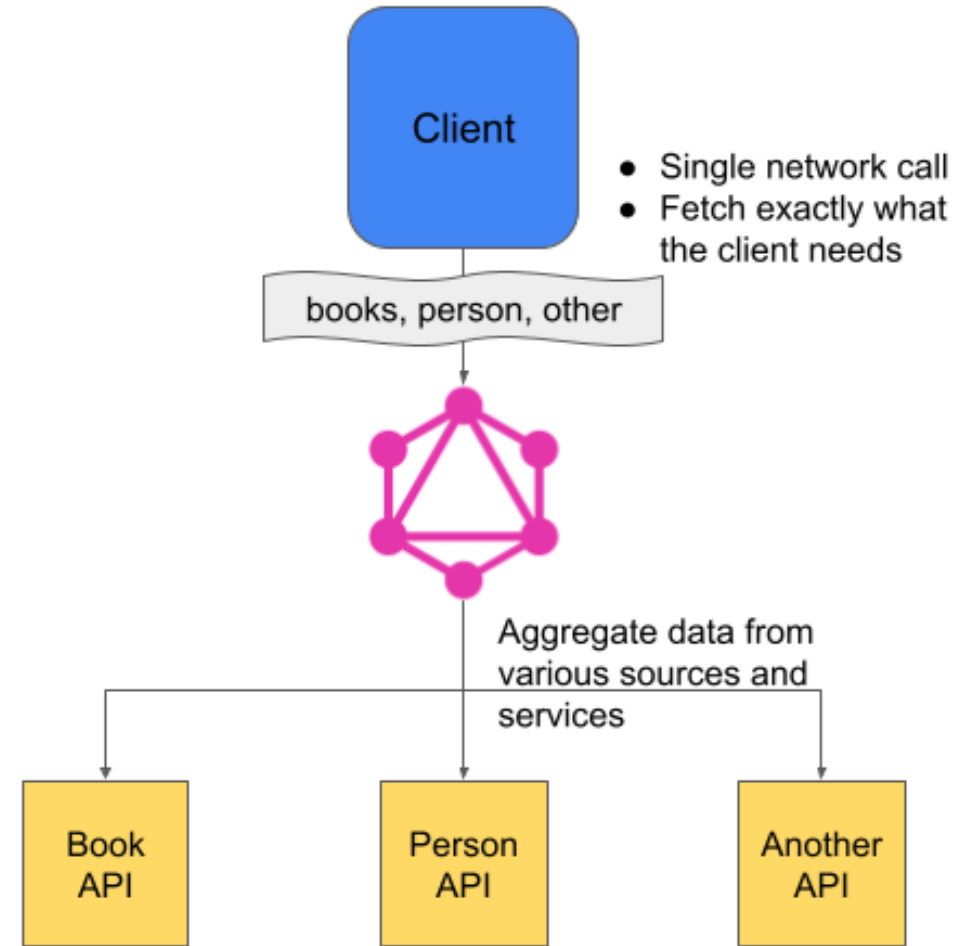
1 request
Desired properties
Desired relationships
Desired shape
Desired property names

Comparaison entre RESTful et GraphQL

REST API



GraphQL API



Comparaison entre RESTful et GraphQL

Tableau comparatif des différences entre RESTful et GraphQL

Caractéristiques	RESTful	GraphQL
Architecture	Basée sur des ressources avec des URIs	Basée sur un schéma de types et de champs
Syntaxe des requêtes	Utilise différentes méthodes HTTP (GET, POST, PUT, DELETE) avec des URIs et des paramètres	Utilise une seule méthode POST avec une syntaxe de requête structurée
Surcharge réseau	Peut entraîner une surcharge du réseau en raison du nombre de requêtes nécessaires pour récupérer des données liées	Minimise la surcharge réseau en permettant aux clients de spécifier exactement les données dont ils ont besoin
Flexibilité des requêtes	Moins flexibles car les Endpoints sont prédéfinis et les données sont récupérées en blocs prédéfinis	Très flexible car les clients peuvent spécifier les données exactes dont ils ont besoin en définissant une requête personnalisée

Comparaison entre RESTful et GraphQL

Tableau comparatif des différences entre RESTful et GraphQL

Caractéristiques	RESTful	GraphQL
Documentation automatique	Nécessite souvent une documentation externe pour découvrir les fonctionnalités de l'API.	Expose automatiquement un schéma décrivant toutes les fonctionnalités de l'API, facilitant la découverte et l'exploration.
Gestion de la version	Souvent nécessite la gestion explicite des versions de l'API via des URI versionnées.	Gère implicitement les changements de schéma grâce à une évolution continue sans nécessité de versions explicites.
Performance	Peut-être plus performant pour les opérations de lecture de données statiques.	Peut-être plus efficace pour les opérations de lecture de données complexes ou pour récupérer des données liées en une seule requête.
Adaptabilité	Bien adapté aux applications avec des données statiques et des opérations CRUD simples.	Convient particulièrement aux applications nécessitant une récupération précise et flexible des données, ainsi qu'à la gestion de l'évolution continue du schéma.



Consommation des services Web

Matiere

localhost:4200

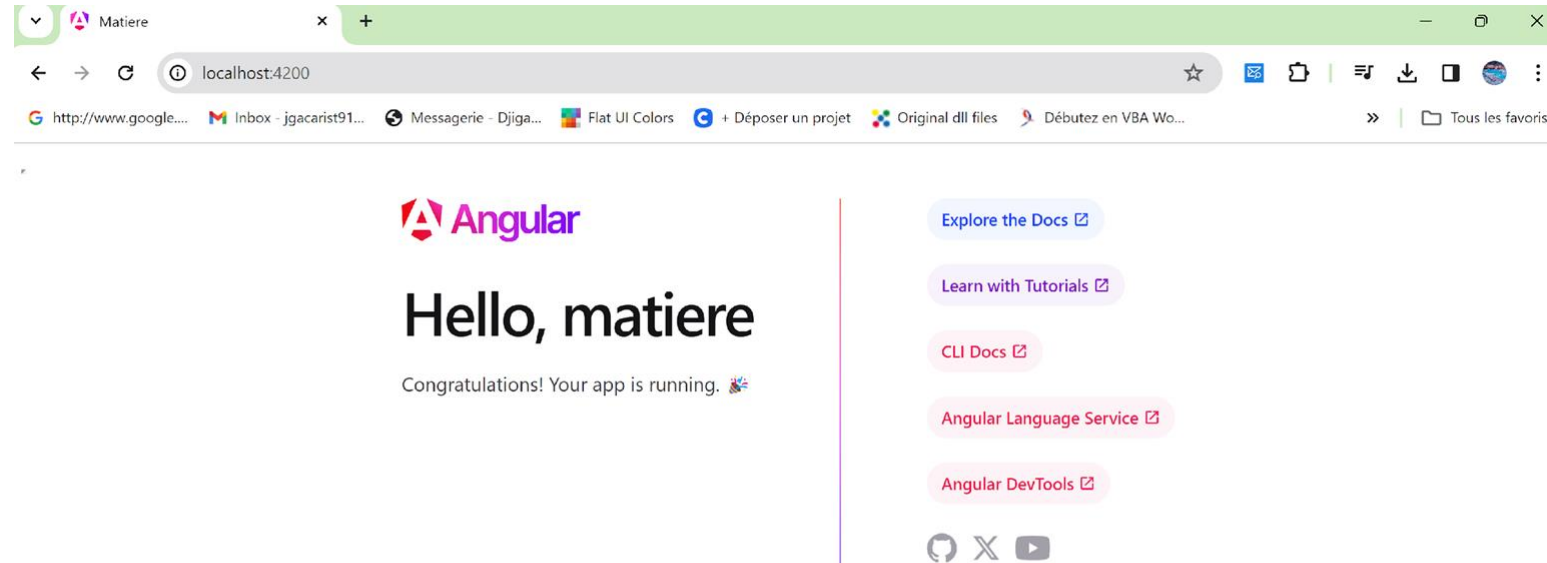
http://www.google.... Inbox - jgacarist91... Messagerie - Djiga...

WebService - Matieres - Sequence 04

ID	Code	Libelle	Crédit
1	CHM789	Chimie	5
3	PHY456	Physique	4
5	MAT123	Maths	6
7	ALG65	ALGEBRE	4
9	LOG001	LOGIQUE	3

Client: Projet Angular

1. Générez un projet angular “Matiere-Client”
2. Ouvrez le projet dans votre IDE
3. Installez angular material
4. Démarrez l'application angular



Client: Projet Angular

1. Créez un service angular “Matiere-Service”
2. Configurez le service pour envoyer une requête GET à l’endpoint “/matieres” défini plutôt dans la séquence 4
NB: Assurez-vous d’injecter le service HttpClient dans votre service Angular
3. Définissez le modèle de données “Matiere” avec une classe TypeScript
4. Créez un composant angular pour afficher la liste des matières
5. Affichez la liste des matières dans le composant

Faites pareil pour ajouter des matières à partir du client Angular



Syntaxe des requêtes GraphQL



Syntaxe GraphQL - Query

GraphQL n'expose toujours qu'un seul endpoint, permettant au client de décider de quelles données il a réellement besoin à partir d'un modèle défini.

```
{  
  Matieres {  
    code  
    libelle  
  }  
}
```

Matieres : champ racine

Code & Libelle (Tout ce qui vient après):

Charge utile

```
 Répons  
{  
  {  
    "code": "CHM789",  
    "libelle": "Chimie"  
  },  
  {  
    "code": "PHY456",  
    "libelle": "Physique"  
  },  
  {  
    "code": "MAT123",  
    "libelle": "Maths"  
  },  
  {  
    "code": "ALG65",  
    "libelle": "ALGEBRE"  
  },  
  {  
    "code": "LOG001",  
    "libelle": "LOGIQUE"  
  }  
}
```

Syntaxe GraphQL - Query

Renvoyez les deux dernières Matieres ajoutées

```
{  
  Matieres (last: 2)  
  {  
    code  
    libelle  
  }  
}
```

Réponses

```
[  
  {  
    "code": "ALG65",  
    "libelle": "ALGEBRE"  
  },  
  {  
    "code": "LOG001",  
    "libelle": "LOGIQUE"  
  }  
]
```

**Zéro ou plusieurs arguments (comme last:2)
peuvent être fournis pour chaque champ dans
GraphQL**

Syntaxe GraphQL - Query

Filtrez les matières en fonction du champ "Code":

```
{  
  Matieres (code:  
    "ALG65") {  
    code  
    libelle  
    credit  
  }  
}
```

Réponses

```
[  
  {  
    "code": "ALG65",  
    "libelle": "ALGEBRE",  
    "credit": 4  
  }  
]
```

**Zéro ou plusieurs arguments (comme last:2)
peuvent être fournis pour chaque champ dans
GraphQL**

Syntaxe GraphQL - Mutation

Dans GraphQL, les mutations sont utilisées pour CUD:

- **Créer de nouvelles données**
- **Mettre à jour les données existantes**
- **Supprimer les données existantes**

La syntaxe des mutations est presque la même que celle des requêtes, mais elles doivent commencer par le mot-clé **“mutation”**.

Syntaxe GraphQL - Mutation

Filtrez les matières en fonction du champ "Code":

```
mutation {  
  add (id:11, code: "ARC100", libelle: "ARCHITECTUE  
LOGICIELLE", credit: 6) {  
    id  
    code  
  }  
}
```

- Crée un nouvel enregistrement
- Renvoie l'id, et le code de la matière nouvellement créée

Syntaxe GraphQL – Subscriptions

- Basés sur des événements, les abonnements sont un moyen de créer et de maintenir une connexion en temps reel au serveur.
- A chaque fois qu'il est déclenché, tous les abonnés de cet événement seront notifiés.

Exemple d'évènements :

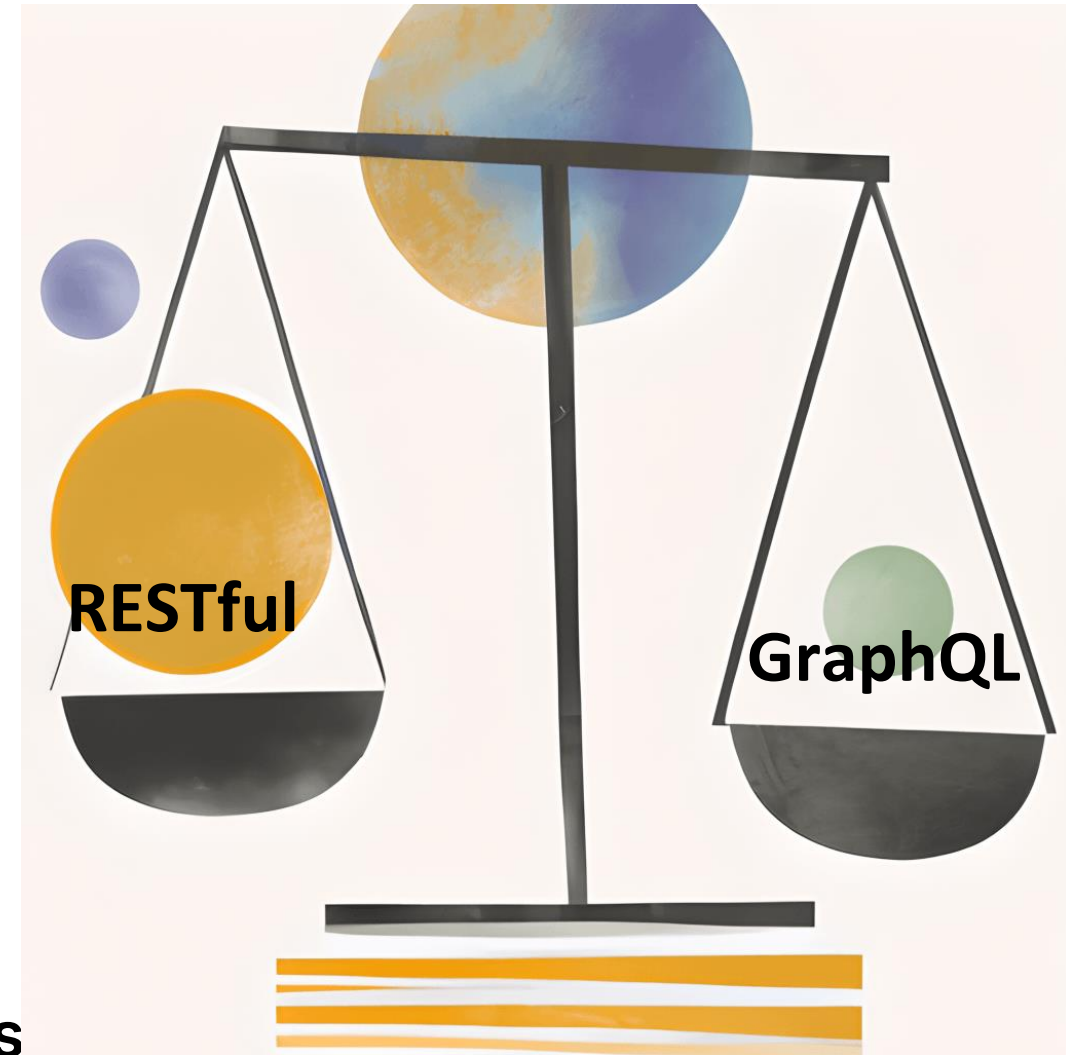
- Ajout d'une nouvelle donnée
- Suppresion d'une donnée existante
- Modification d'une donnée existante

Avantages de GraphQL



Avantages de GraphQL

1. **Récupération précise des données:** Les clients peuvent spécifier exactement les données dont ils ont besoin, évitant ainsi le surchargement de données inutiles.
2. **Surcharge minimale:** possibilité de récupérer toutes les données nécessaires en une seule requête.
3. **Flexibilité des requêtes:** Les clients peuvent formuler des requêtes flexibles pour récupérer des données de différentes sources en une seule requête.



Avantages de GraphQL (Su

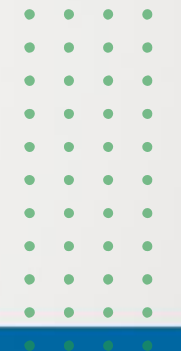
4. Gestion automatique du schéma: Il expose automatiquement un schéma décrivant toutes les fonctionnalités de l'API, facilitant ainsi la découverte et l'exploration.



- **Evolution continue du schéma:** permet d'évoluer continuellement le schema sans nécessiter de versions explicites, offrant ainsi une gestion flexible de l'évolution des données.
- **Meilleure expérience de développement**

Webographie

- **Cours 1FullStack2201 Langages et Frameworks Backend 1- Cours Spring Boot du Pr Ousmane SALL**
- <https://www.jobtrack.com.au/site/JT/blog/article/rest-define/>
- https://www.researchgate.net/figure/Stateless-and-stateful-data-plane-Adapted-from-132-133_fig1_354302313
- <https://www.xenonstack.com/insights/stateful-and-stateless-applications>
- <https://www.howtographql.com/basics/1-graphql-is-the-better-rest/>
- <https://docs.kaleido.io/kaleido-services/baf/oauth/>



MERCI

